

Proyecto Integrador de Aprendizaje Automático

Predicción de falla cardíaca con un flujo MLOps en entorno local · Evidencias por etapa

Donnys Torres Lozano · Maestría en Analítica de Datos, Universidad del Norte · Machine Learning (Prof. Lihki Rubio) · 9 de octubre de 2026
Repositorio: <https://github.com/DTL-DA/heart-disease-mlops>

Este documento reúne, para cada etapa de la sección 10.20, qué pide el enunciado, en qué archivo está y qué resultado se obtuvo. El modelo final es un RandomForest dentro de un Pipeline, con AUC de 0,931 en validación cruzada y en prueba. Se sirve con FastAPI, se containeriza con Docker, se despliega en Minikube, se valida con GitHub Actions y se monitorea con Evidently.

Etapa	Entregable	Resultado clave
0	Estructura modular	Carpetas del enunciado, más <code>src/</code> , <code>tests/</code> y <code>monitoring/</code> de apoyo
1	Análisis, fuga de datos y ranking	7 modelos con Pipeline y GridSearchCV; RandomForest primero, con AUC de 0,931 en validación cruzada
2	Modelado con validación segura	AUC de prueba 0,931; exactitud 0,880; sensibilidad 0,902
3	FastAPI y Docker	Imagen <code>heart-api:latest</code> ; <code>/predict</code> responde 0,9767 para el paciente de ejemplo
4	Kubernetes (Minikube)	Pod <code>heart-model</code> en <code>1/1 Running</code> ; mismo resultado por el servicio
5	Integración continua	<code>flake8</code> y 6 pruebas de <code>pytest</code> en verde en cada <code>push</code>
6	Monitoreo de deriva	0 de 11 variables con deriva (Evidently)

0 Estructura

Qué pide el enunciado. Crear una estructura modular para alojar códigos, cuadernos, API, despliegues y CI/CD.
Dónde está. Raíz del repositorio.

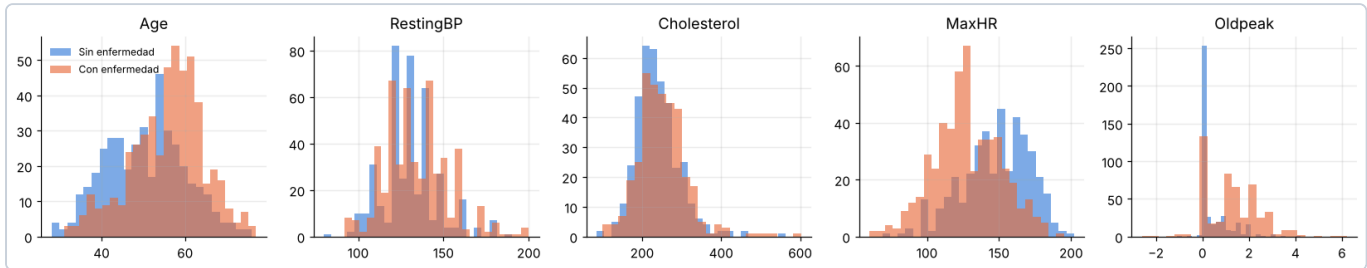
```
heart-disease-mlops/
├── app/                api.py, model.joblib
├── data/              heart.csv
├── docker/            Dockerfile, requirements.txt
├── k8s/               deployment.yaml, service.yaml
├── notebooks/         1_model_leakage_demo.ipynb, 2_model_pipeline_cv.ipynb
├── .github/workflows/ ci.yml
├── src/               heart_ml.py                (funciones reutilizables)
├── monitoring/        drift_report.py
├── tests/             test_api.py
├── evidencias/        capturas y este documento
├── train.py
├── requirements-dev.txt
├── .flake8
├── drift_report.html
├── model.joblib
└── README.md
```

Resultado. La estructura sigue la propuesta del enunciado. Se agregaron `src/` (funciones reutilizables), `tests/` (pruebas que ejecuta el CI), `monitoring/` (script de Evidently) y `train.py`, porque el enunciado pide separar el código en funciones reutilizables y ejecutar `pytest tests/`.

1 Análisis exploratorio, preprocesamiento, fuga de datos y ranking de modelos

Qué pide el enunciado. Explorar los datos, tratar valores nulos, evitar la fuga de datos y preparar los datos. Implementar los modelos vistos dentro de un `Pipeline` con `GridSearchCV` y presentar un ranking comparativo con AUC y precisión.

Dónde está. `notebooks/1_model_leakage_demo.ipynb` y `src/heart_ml.py`.



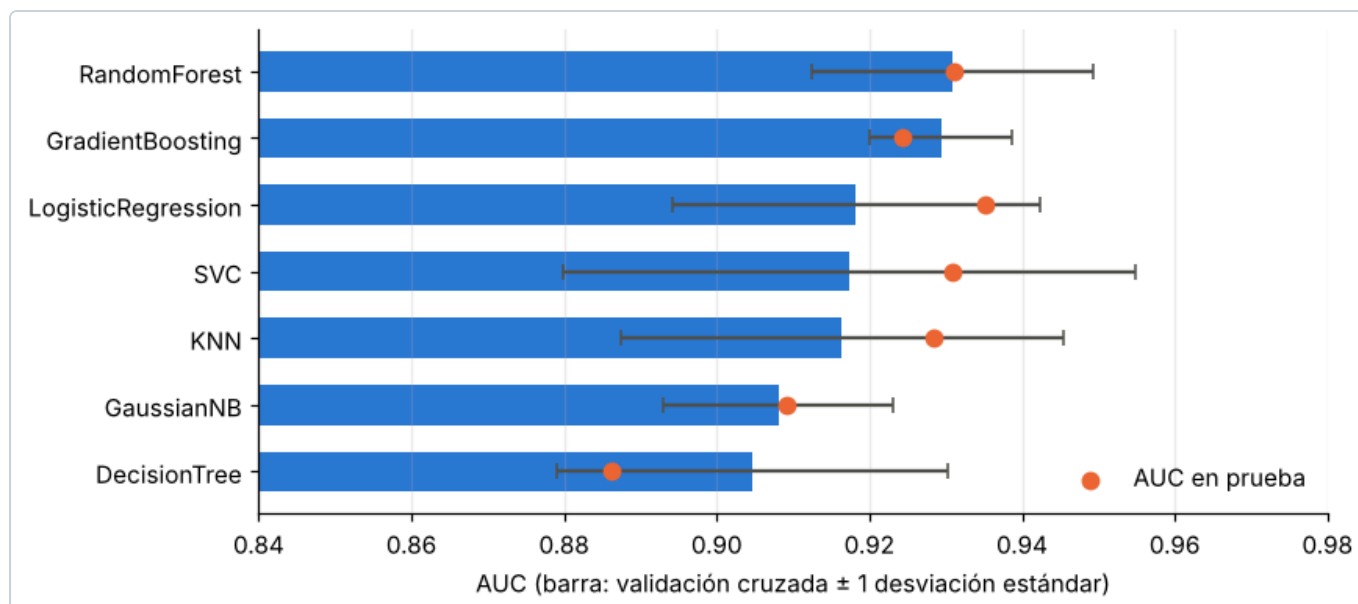
Distribución de las variables numéricas según la clase (cuaderno 1)

Fuga de datos

Caso	AUC validación cruzada	AUC prueba
A. Variable que filtra el objetivo (<code>leaky_feature</code>)	n. a.	1,000
B. Preprocesamiento con fuga (ajustado con todos los datos)	0,917	0,926
B. Sin fuga (dentro del <code>Pipeline</code>)	0,917	0,931

Ranking de los 7 modelos

Puesto	Modelo	AUC validación cruzada	AUC prueba	Exactitud prueba	Precisión prueba	Sensibilidad prueba
1	RandomForest	0,931 ± 0,018	0,931	0,880	0,885	0,902
2	GradientBoosting	0,929 ± 0,009	0,924	0,886	0,893	0,902
3	LogisticRegression	0,918 ± 0,024	0,935	0,880	0,877	0,912
4	SVC	0,917 ± 0,037	0,931	0,853	0,864	0,873
5	KNN	0,916 ± 0,029	0,928	0,870	0,890	0,873
6	GaussianNB	0,908 ± 0,015	0,909	0,875	0,891	0,882
7	DecisionTree	0,905 ± 0,026	0,886	0,842	0,848	0,873



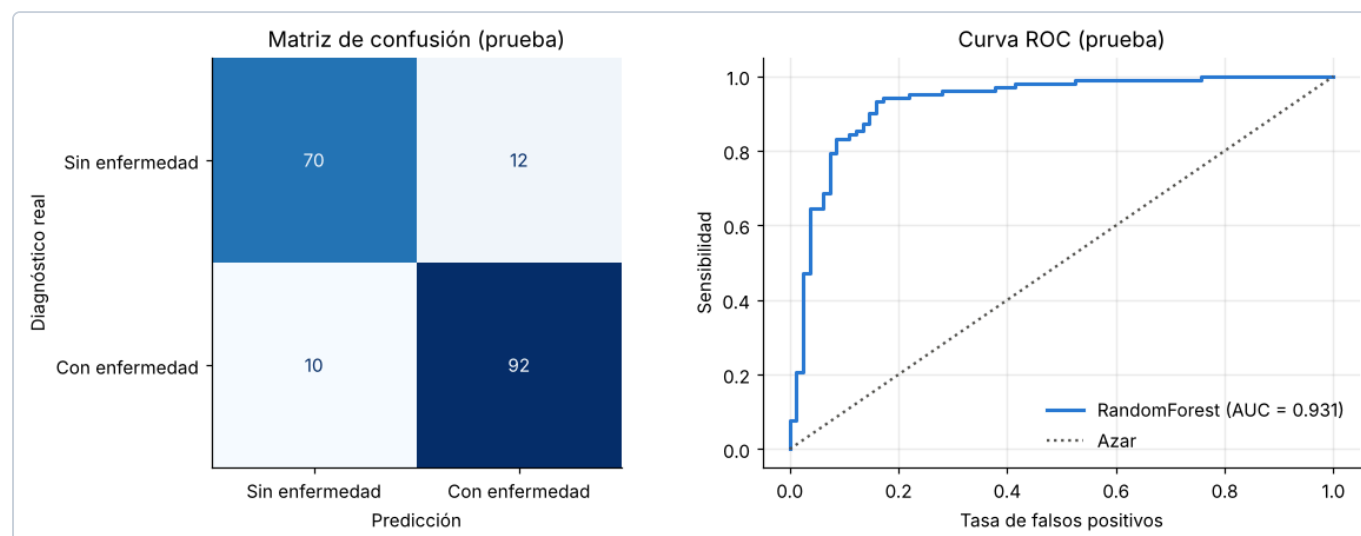
AUC en validación cruzada por modelo, con barras de error (cuaderno 1)

Resultado. 172 registros con colesterol en cero se trataron como faltantes (imputación con la mediana dentro del Pipeline) y se eliminó 1 registro con presión en cero, de modo que quedan 917 pacientes para modelar. Los siete modelos usan Pipeline y GridSearchCV de 5 pliegues con el AUC como criterio. Los cinco primeros están separados por 0,015 de AUC, menos que la desviación entre pliegues, así que son estadísticamente equivalentes; se elige RandomForest por el AUC de validación cruzada y no por el de prueba.

2 Modelado con validación segura

Qué pide el enunciado. Dividir los datos antes del escalado, implementar `Pipeline` con `GridSearchCV` y evaluar con matriz de confusión, curva ROC y AUC.

Dónde está. `notebooks/2_model_pipeline_cv.ipynb`.



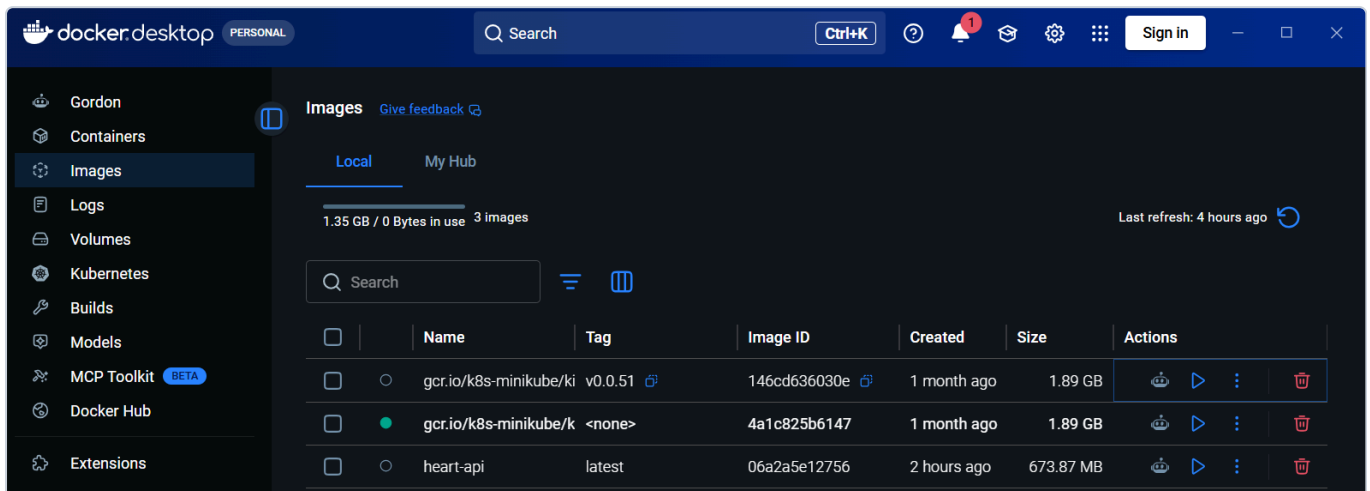
Matriz de confusión y curva ROC en el conjunto de prueba (cuaderno 2)

Resultado. En prueba (184 pacientes) el modelo obtiene AUC 0,931, exactitud 0,880, precisión 0,885 y sensibilidad 0,902. La matriz muestra 70 verdaderos negativos, 12 falsos positivos, 10 falsos negativos y 92 verdaderos positivos. El AUC de prueba coincide con el de validación cruzada ($0,931 \pm 0,018$). En entrenamiento el AUC es 0,996: hay un sobreajuste moderado, propio del RandomForest, por lo que se reporta 0,931 como desempeño esperado.

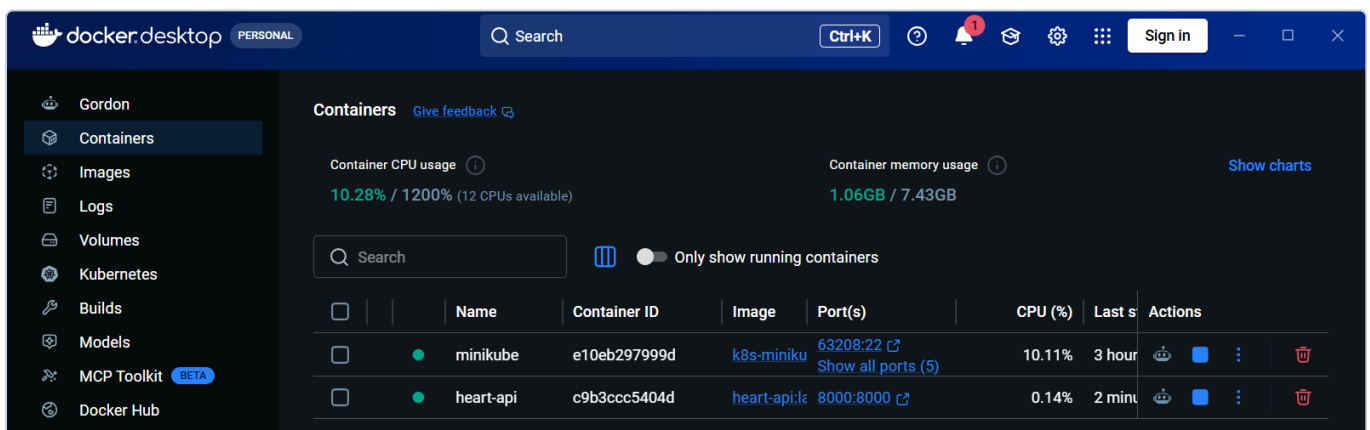
3 Despliegue con FastAPI y Docker

Qué pide el enunciado. Servir el modelo mediante una API REST y contenerizar la aplicación con Docker.

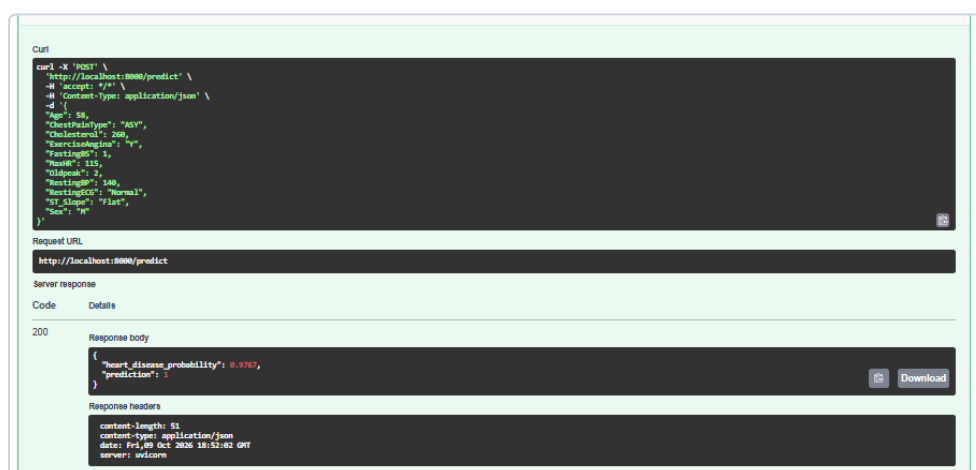
Dónde está. `app/api.py`, `docker/Dockerfile` y `docker/requirements.txt`.



Docker Desktop: imagen heart-api:latest construida con docker build



Contenedor heart-api en ejecución, puerto 8000



POST /predict contra http://localhost:8000: código 200 y probabilidad

Resultado. La imagen heart-api:latest se construyó con `docker build` y se ejecutó con `docker run -p 8000:8000`. La API valida los datos del paciente y devuelve una probabilidad de 0,9767 y la predicción 1 para el paciente de ejemplo.

4 Despliegue en Kubernetes local

Qué pide el enunciado. Crear los manifiestos `Deployment` y `Service` para implementar el modelo localmente.

Dónde está. `k8s/deployment.yaml` y `k8s/service.yaml`.

```
PS C:\Users\donov\OneDrive\Escritorio\1. Uninorte\3er Semestre\Machine Learning\Tarea 2> kubectl get pods,svc,deploy
>>
NAME                                     READY   STATUS    RESTARTS   AGE
pod/heart-model-689f98795b-6m9cq        1/1     Running   0           157m

NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/heart-service               LoadBalancer  10.99.218.233    <pending>     80:32613/TCP     3h41m
service/kubernetes                  ClusterIP      10.96.0.1        <none>        443/TCP          3h42m

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-model         1/1     1             1           3h41m
PS C:\Users\donov\OneDrive\Escritorio\1. Uninorte\3er Semestre\Machine Learning\Tarea 2> 
```

kubectl get pods,svc,deploy: el pod heart-model en 1/1 Running



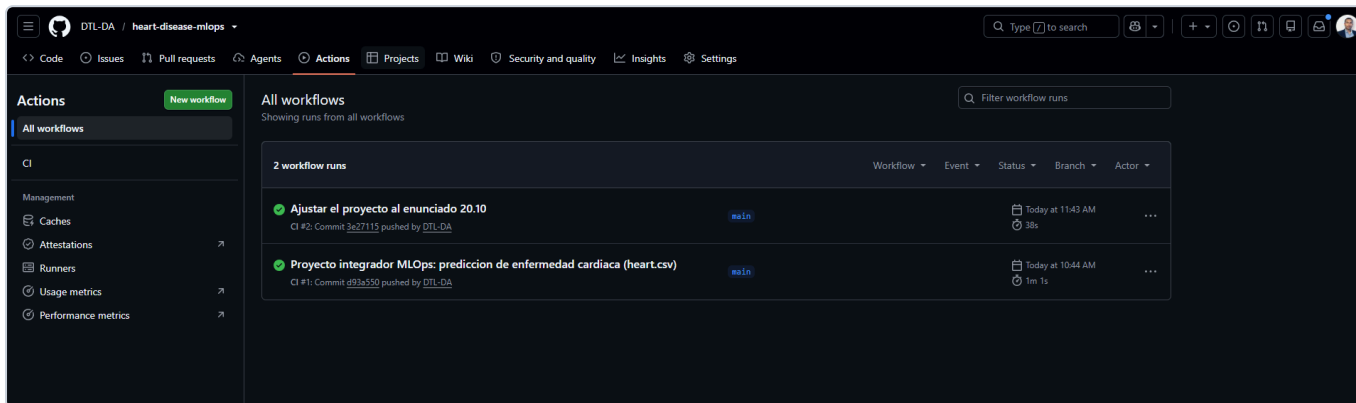
POST /predict a través del servicio de Minikube: código 200 y probabilidad

Resultado. Minikube corre sobre Docker. La imagen local se cargó con `minikube image load` y los manifiestos se aplicaron con `kubectl apply`. El pod `heart-model` queda en `1/1 Running` y el servicio, abierto con `minikube service heart-service --url`, devuelve el mismo resultado (0,9767).

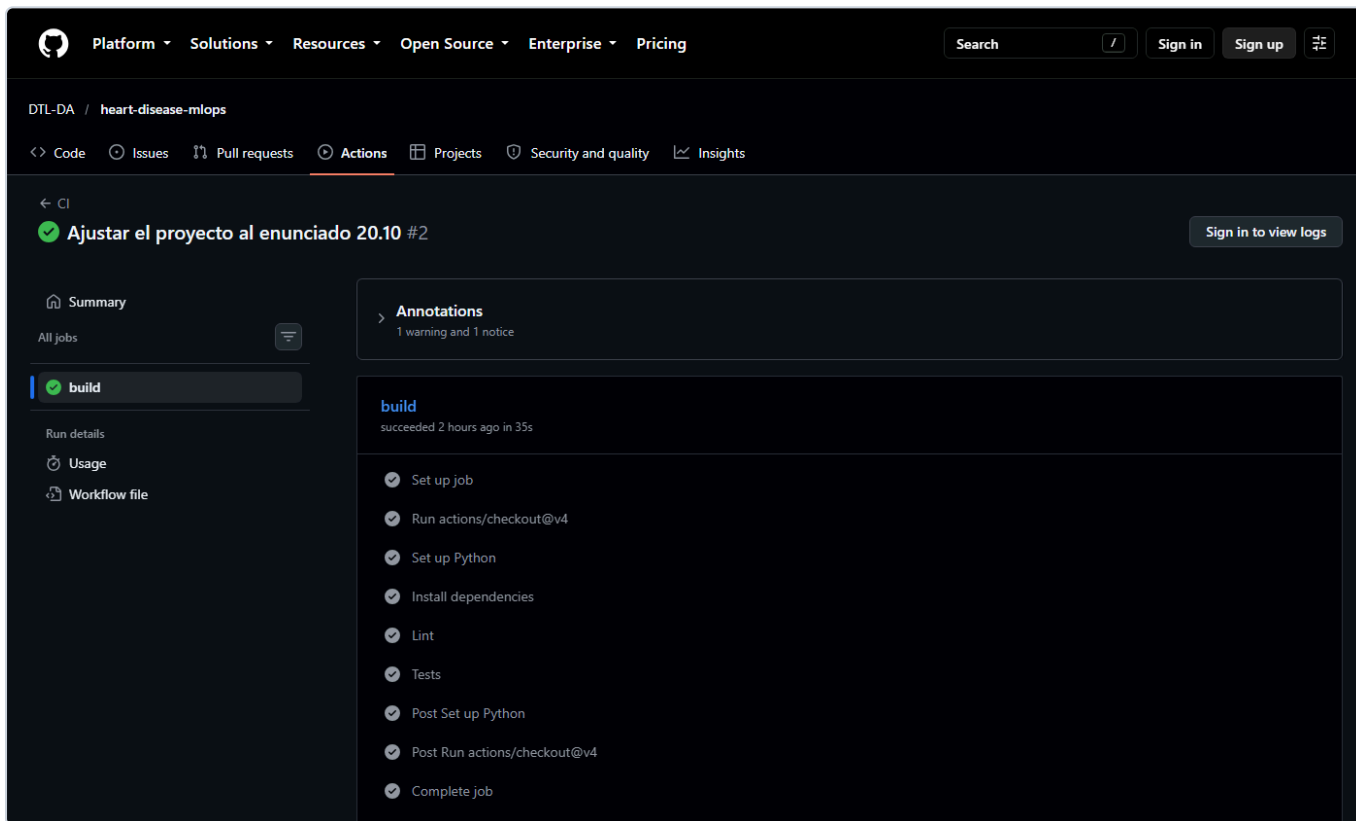
5 Integración continua con GitHub Actions

Qué pide el enunciado. Agregar pruebas automáticas, revisión de estilo (linting) y validación del código en cada pulsación.

Dónde está. `.github/workflows/ci.yml` y `tests/test_api.py`.



Pestaña Actions del repositorio: las dos corridas en verde



Pasos de la corrida #2: Lint y Tests en verde (vista pública, sin iniciar sesión)

Resultado. Cada push ejecuta `flake8` y `pytest` (6 pruebas de la API) en `ubuntu-latest` con Python 3.13. Las dos corridas del repositorio terminan en verde.

6 Monitoreo de deriva de datos

Qué pide el enunciado. Generar informes de deriva de datos entre entrenamiento y predicción.

Dónde está. `monitoring/drift_report.py` y `drift_report.html`.



Reporte de Evidently: resumen y las 11 variables



Detalle: distribuciones de referencia y actuales, prueba estadística y valor p

Resultado. Evidently compara entrenamiento (referencia) con prueba (actual). No se detecta deriva en ninguna de las 11 variables; la más cercana al umbral de 0,05 es `Cholesterol`, con $p = 0,067$. Es el resultado esperado, porque ambos conjuntos salen de la misma partición estratificada, y sirve como línea base. En producción, los datos actuales se reemplazan por los registros que recibe la API.

Cómo reproducir el proyecto

Los comandos siguen el orden de las etapas y se ejecutan desde la raíz del repositorio. La tabla «etapa → archivo → evidencia» está en el [README.md](#).

```
pip install -r requirements-dev.txt
python train.py
docker build -t heart-api -f docker/Dockerfile .
docker run -p 8000:8000 heart-api
minikube start
minikube image load heart-api:latest
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
flake8 app/ src/ tests/ train.py
pytest tests/ -v
python monitoring/drift_report.py
```

Etapas 1 y 2: entrena y exporta el modelo
Etapa 3
Etapa 4
Etapa 5
Etapa 6